INSIDE STLC

# PLAYWRIGHT FRAMEWORK HEALTH CHECK 2026

Your first test passing proves Playwright works. This checklist asks whether your project will still work at test 50.

**THE 5-MINUTE TEST: Score each break below. If changing one UI element, one user or one laptop can cripple your suite, the problem is structure - not coding talent.**

| BREAK 1 | LOCATORS EVERYWHERE | YOUR CHECK |
|---|---|---|
| **Symptom**<br>The same selector or UI detail is repeated across many tests. | **Why it fails at scale**<br>One UI change creates dozens of edits and failures. AI can generate copies faster, but that only multiplies maintenance. | [ ] One UI label change breaks multiple tests<br>[ ] Selectors are copied between files<br>[ ] Fixing a locator means search-and-replace |
| **Health rule** | **If one shared UI change requires edits in more than one test, your structure needs attention.** | **Score:** 0 / 1 / 2 / 3 |

| BREAK 2 | TEST DATA WELDED INTO TESTS | YOUR CHECK |
|---|---|---|
| **Symptom**<br>Emails, passwords, users and scenario data live directly inside test files. | **Why it fails at scale**<br>You duplicate whole tests just to change data. Parallel execution becomes harder and test intent gets buried in setup. | [ ] Credentials are hard-coded<br>[ ] Same test copied for different users<br>[ ] Data changes require editing test logic |
| **Health rule** | **One test should be able to exercise multiple relevant data scenarios without cloning the test itself.** | **Score:** 0 / 1 / 2 / 3 |

| BREAK 3 | NO HOME IN GIT / CI | YOUR CHECK |
|---|---|---|
| **Symptom**<br>The suite only runs when you manually start it on your laptop. | **Why it fails at scale**<br>No history, no repeatable feedback and no protection when you are busy or unavailable. Automation becomes a personal script collection. | [ ] Tests live only locally<br>[ ] No version history / review<br>[ ] No automated run on push, schedule or pipeline |
| **Health rule** | **If the tests stop protecting the product when your laptop closes, the automation system is incomplete.** | **Score:** 0 / 1 / 2 / 3 |

| TOTAL SCORE | 0-2: HEALTHY START | 3-5: STRUCTURE WARNING | 6-9: MAINTENANCE DEBT |
|---|---|---|---|
| **____ / 9** | Keep the suite simple. Add structure only as real repetition and pain appear. | Refactor before adding lots more tests. Identify the repeated change points first. | Stop scaling temporarily. Fix shared locators, separate data and establish Git/CI before test 51. |

| THE RIGHT PROGRESSION | THE PRINCIPLE |
|---|---|
| 1 valuable test -> a few stable journeys -> feel genuine duplication/maintenance pain -> centralise shared UI behaviour -> separate reusable test data -> Git -> CI/CD -> scale deliberately. | Framework complexity should solve a problem you actually have - not a problem a tutorial says you might have. |

**Inside STLC Academy:** Practical Test Automation with Playwright - framework architecture, test data, Git, CI/CD, portfolio project and AI prompt packs.
insidestlcacademy.com | Use this checklist as a practical diagnostic. The right framework design depends on your product, team, risk and delivery context.